

Zero-Shot Detection of LLM-Generated Text using Token Cohesiveness

Anonymous ACL submission

Abstract

The increasing capability and widespread usage of large language models (LLMs) highlight the desirability of automatic detection of LLM-generated text. Zero-shot detectors, due to their training-free nature, have received considerable attention and notable success. In this paper, we identify a new feature, token cohesiveness, that is useful for zero-shot detection, and we demonstrate that LLM-generated text tends to exhibit higher token cohesiveness than human-written text. Based on this observation, we devise TOCSIN, a generic dual-channel detection paradigm that uses token cohesiveness as a plug-and-play module to improve existing zero-shot detectors. To calculate token cohesiveness, TOCSIN only requires a few rounds of random token deletion and semantic difference measurement, making it particularly suitable for a practical black-box setting where the source model used for generation is not accessible. Extensive experiments with two state-of-the-art base detectors on various datasets, source models, and evaluation settings demonstrate the effectiveness and generality of the proposed approach.

1 Introduction

The past few years have witnessed tremendous advances in Large Language Models (LLMs). These models such as ChatGPT (OpenAI, 2022), PaLM (Chowdhery et al., 2023), GPT-4 (OpenAI, 2023) can now generate text of supreme quality, demonstrating exceptional performance in various fields like question answering, news reporting, and story writing. The increasing capability of LLMs to produce human-like text at high efficiency, however, also raises concerns about their misuse for malicious purposes, e.g., phishing (Panda et al., 2024), disinformation (Jiang et al., 2024), and academic dishonesty (Perkins, 2023). The effective detection of LLM-generated text therefore becomes a vital principle to ensure the responsible use of LLMs.

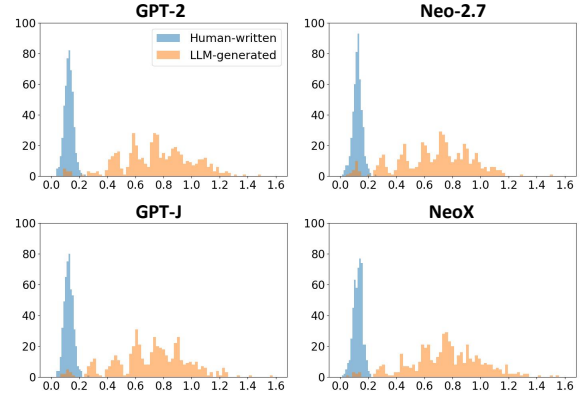


Figure 1: Distribution of token cohesiveness between 500 human-written articles and 500 LLM-generated articles. Human-written articles are sampled from XSum (Narayan et al., 2018), and LLM-generated articles are produced by prompting four source models with the first 30 tokens of each human-written article. The calculation of token cohesiveness will be detailed in Section 3.2.

LLM-generated text detection is typically formulated as a binary classification task, i.e., to classify whether a piece of text is generated by LLMs or written by human (Tang et al., 2024). Current solutions to the task can be roughly divided into two categories: supervised classifiers (Ippolito et al., 2020; Fagni et al., 2021) and zero-shot classifiers (Gehrmann et al., 2019; Mitchell et al., 2023; Zhu et al., 2023). Supervised classifiers are trained to learn the discrepancies between LLM-generated and human-written text from labeled data, and are therefore prone to overfit their specific training domains and may have poor generalization ability (Pu et al., 2023; Wang et al., 2023b). Zero-shot classifiers, which extract diverse features discriminating between the two kinds of text and directly perform thresholding on these features without training, are more immune to domain-specific degradation and usually generalize better (Zhu et al., 2023).

In this paper we introduce a new feature, namely, token cohesiveness, for zero-shot LLM-generated

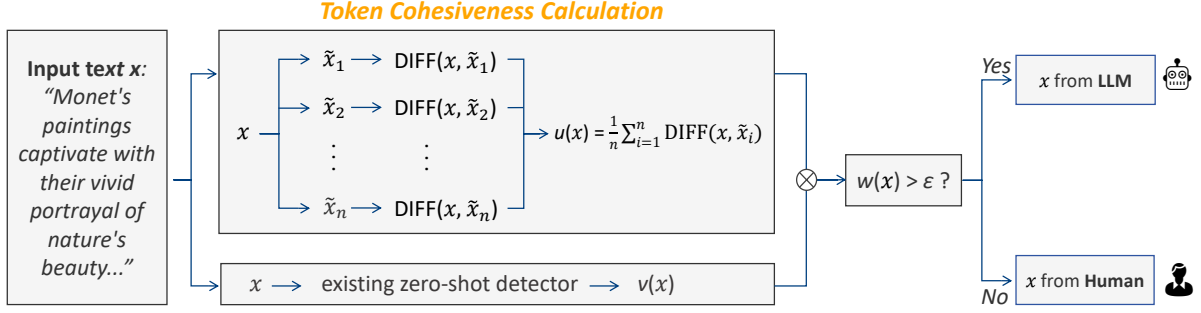


Figure 2: Overview of TOCSIN. The input text x is fed into the upper channel to calculate token cohesiveness $u(x)$, and the lower channel to produce raw prediction $v(x)$. The two scores are then combined into $w(x)$, and if the combination exceeds a predefined threshold ϵ , the text x is categorized as LLM-generated.

text detection. Token cohesiveness is defined as the expectation of semantic difference between input text x and its copy $\tilde{x}$ after randomly removing a small proportion of tokens. It essentially measures how closely the removed tokens are semantically related to other tokens in the input text. Our key assertion is that **LLM-generated text generally exhibits higher token cohesiveness than human-written text**. This is because LLMs use the causal self-attention mechanism (Vaswani et al., 2017) to generate text, requiring the generation of each token to be conditioned on all its preceding tokens, which would naturally foster a closer relationship among tokens, thus increasing token cohesiveness. In contrast, humans tend to write text more freely with no such explicit restriction, which potentially results in a looser relationship among tokens, thus reducing token cohesiveness. We empirically verify this assertion, and find that it holds true across a diverse body of LLMs, as illustrated in Figure 1.

Upon the above finding, we devise **TOCSIN**, a novel paradigm that uses **Token CoheSiveNess** as an enhancement to improve zero-shot detection of LLM-generated text. TOCSIN is a dual-channel detector, with one channel equipped with an existing zero-shot detector, and the other channel equipped with a token cohesiveness calculation module. The overall procedure is sketched in Figure 2. Specifically, given a piece of text to be detected, we create multiple copies, with each copy randomly removing a specific proportion of tokens from the input text. We calculate the average semantic difference, or more precisely, the average negative BART score (Yuan et al., 2021) between the input text and these copies as the token cohesiveness score. Meanwhile, we feed the input text into an existing zero-shot detector to produce a raw prediction score. After that, we combine the two scores and perform threshold-

ing on the combination to make the final decision. In this way, we can integrate token cohesiveness as a supplement to favor more accurate detection.

As an enhancement to existing zero-shot detectors, TOCSIN enjoys several merits. (1) **General applicability**: The dual-channel solution of TOCSIN is quite generic, allowing token cohesiveness to be used as a plug-and-play module in a variety of detectors. (2) **Low additional time cost**: TOCSIN keeps the existing detector channel unchanged, and the additional time cost mainly comes from token cohesiveness calculation, which only involves several rounds of random token deletion and BART score computation, and is highly time efficient. (3) **Low additional space cost**: The only additional space cost comes from loading BART (Lewis et al., 2020) (or more precisely, BART-base) for token cohesiveness calculation, which is relatively small compared to those feature extractors (a.k.a. surrogate models) used in existing zero-shot detectors.

To rigorously evaluate the effectiveness and generality of TOCSIN, we apply it to two current state-of-the-art zero-shot detectors, LRR (Su et al., 2023) and Fast-DetectGPT (Bao et al., 2024), and conduct extensive experiments on four diversified datasets, with LLM-generated passages produced by seven different source models ranging from GPT-2 (Radford et al., 2019) to GPT-4 (OpenAI, 2023). Experimental results demonstrate consistent and meaningful improvements over the two detectors in both white-box and black-box settings.

Our main contributions in this paper are three-fold: (1) unveiling and validating a new hypothesis that LLM-generated text exhibits higher token cohesiveness than human-written text, (2) proposing a novel and generic framework that uses token cohesiveness to improve zero-shot detection of LLM-generated text, and (3) achieving new best detection

accuracy in both white-box and black-box settings compare to existing zero-shot detectors.

2 Related Work

The increasingly powerful LLMs (Radford et al., 2019; OpenAI, 2022, 2023; Chowdhery et al., 2023; Touvron et al., 2023), though demonstrating excellent performance on various language-related tasks, raise numerous ethical concerns, drawing extensive attention to automatic detection of LLM-generated text (Guo et al., 2023; Li et al., 2023b).

LLM-generated text detection is typically formulated as a binary classification task, with current solutions roughly categorized into supervised classifiers and zero-shot classifiers. Supervised classifiers are those trained with statistical features (Solaiman et al., 2019; Ippolito et al., 2020; Wu et al., 2023; Verma et al., 2023) or neural representations (Uchendu et al., 2020; Bakhtin et al., 2019; Zhong et al., 2020; Bhattacharjee et al., 2023; Wang et al., 2024) to discriminate LLM-generated and human-written text, wherein a popular trend is to directly fine-tune a pre-trained language model like BERT (Devlin et al., 2019) or RoBERTa (Liu et al., 2019) for the classification task (Zellers et al., 2019; Rodriguez et al., 2022; Mitrović et al., 2023; Chen et al., 2023). These supervised classifiers, though achieving excellent performance on their training domains, require periodic retraining to adapt to new LLMs, and often exhibit a tendency to overfit their training data (Pu et al., 2023).

Zero-shot classifiers are entirely training-free and often show better generalization ability. Their key ideas are to extract various statistical features, e.g., likelihood (Hashimoto et al., 2019), perplexity (Lavergne et al., 2008), normalized log-rank perturbation (Su et al., 2023), probability and conditional probability curvatures (Mitchell et al., 2023; Bao et al., 2024), and perform thresholding on these features to discern LLM-generated and human-written text. Such features can be collected from the source LLMs themselves (the white-box setting) or from surrogate models (the black-box setting). In this paper, we propose a new feature called token cohesiveness to improve existing zero-shot detectors. The computation of the new feature does not rely on the source LLMs, making it particularly suitable for the black-box setting.

Recently there emerge some black-box zero-shot detectors based on LLM rewriting (Zhu et al., 2023; Yang et al., 2024). The idea is to rewrite a passage

using another LLM, typically ChatGPT, and then assess the similarity or overlap between the original and recomposed text. Passages with larger similarity or overlap will be regarded as LLM-generated. Our approach similarly adheres to the principle of text reconstruction. But it considers only random token deletion and does not require additional API calls, thus is more efficient and economical.

Besides binary classification, some recent studies consider more challenging LLM-generated text detection tasks, e.g., detecting mixtures of human-written and LLM-modified text (Wang et al., 2023a) and tracing the origin of text generation (Li et al., 2023a). These topics are out of the scope of this paper and will be studied as future work.

3 Methodology

This section presents TOCSIN, a novel paradigm to improve zero-shot detection of LLM-generated text. Below we formally define the task in Section 3.1, then illustrate our key assumption in Section 3.2, followed by the detailed approach in Section 3.3.

3.1 Problem Formulation

We study zero-shot LLM-generated text detection, which is formulated as a binary classification problem. Given a piece of text, or candidate passage x , the goal is to discern whether x is human-written or generated by a source LLM. The problem is zero-shot in the sense that we do not assume access to any human-written or LLM-generated samples to perform detection.

The method we propose is an enhancement to existing zero-shot detectors. Besides the requirements of the base detector, it also makes use of a semantic similarity measurement model, e.g., BART score (Yuan et al., 2021), to calculate token cohesiveness of the candidate passage (see Section 3.2 for details). This model is relatively small and is used off-the-shelf without any fine-tuning.

3.2 Key Assumption

We introduce a new feature, token cohesiveness, to distinguish between LLM-generated and human-written text, and our key assumption is that samples from a source LLM typically exhibit higher token cohesiveness than human-written text. Below we give a formal definition of token cohesiveness, and state the assumption formally as the token cohesiveness disparity hypothesis. We also provide an empirical verification of the hypothesis.

Definition (Token Cohesiveness). Given a candidate passage x , let $\tilde{x}$ denote a random copy created by removing a small proportion of tokens from x . The token cohesiveness of x is then defined as the expectation of semantic difference between x and $\tilde{x}$, i.e., $u(x) \triangleq \mathbb{E}(\text{DIFF}(x, \tilde{x}))$, where $\text{DIFF}(\cdot, \cdot)$ is a semantic difference metric, and $\mathbb{E}(\cdot)$ the expectation operation.

Token cohesiveness essentially measures the semantic closeness among tokens in the passage. The closer the tokens are semantically related to each other, the higher the token cohesiveness would be. We argue that there is a gap in the token cohesiveness between LLM-generated and human-written text. For LLM-generated text, each token is generated based on all its preceding tokens. This would naturally foster a closer relationship among tokens, thereby leading to higher token cohesiveness. But for human-written text, there is no such an explicit restriction about token generation, potentially resulting in a looser relationship among tokens, and consequently lower token cohesiveness. We formalize the assertion into a token cohesiveness disparity hypothesis.

Hypothesis (Token Cohesiveness Disparity). Let $\mathcal{P}_{LLM}$ denote the distribution of LLM-generated text, and $\mathcal{P}_{Human}$ the distribution of human-written text. Then the token cohesiveness $u(x)$ tends to be higher for samples $x \sim \mathcal{P}_{LLM}$, while lower for samples $x \sim \mathcal{P}_{Human}$.

We empirically verify the hypothesis in an automated manner. Specifically, as in prior work (Bao et al., 2024; Mitchell et al., 2023), we use 500 news articles randomly sampled from XSum (Narayan et al., 2018) as human-written data, and use the output of four different LLMs when prompted with the first 30 tokens of each human-written article as LLM-generated data. For each article, to calculate its token cohesiveness, we create 10 copies, each randomly deleting 1.5% tokens from the original article.¹ We use negative BART score (Yuan et al., 2021) as the semantic difference metric $\text{DIFF}(\cdot, \cdot)$, and approximate the expectation $\mathbb{E}(\cdot)$ with the average of the 10 copies. Figure 1 shows the results, which reveal that the distribution of token cohesiveness differs significantly for LLM-generated and human-written data. LLM-generated samples do tend to have larger token cohesiveness.

¹We later show in Section 4.4 how the number of copies and deletion proportion affect detection performance.

Algorithm 1 TOCSIN LLM-generated text detection

Input: passage x , base detector $v(\cdot)$, semantic difference metric $\text{DIFF}(\cdot, \cdot)$, decision threshold ϵ
Output: True – probably LLM-generated, False – probably human-written
1: create copies $\{\tilde{x}_i\}_{i=1}^n$ via random token deletion
2: calculate token cohesiveness $u(x) = \sum_{i=1}^n \frac{\text{DIFF}(x, \tilde{x}_i)}{n}$
3: get raw prediction $v(x)$ from base detector
4: combine into new prediction $w(x) = e^{u(x)} \times v(x)$
5: **return** $w(x) > \epsilon$

3.3 Detailed Approach

Based on the above findings, we devise TOCSIN, a generic dual-channel detection paradigm that uses token cohesiveness as a plug-and-play module to improve existing zero-shot detectors. The overall architecture of TOCSIN is sketched in Figure 2.

Specifically, given a passage x , we first feed it into one channel to calculate its token cohesiveness. This channel creates n copies $\{\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_n\}$ for the input x , with each copy randomly deleting a certain proportion (denoted as ρ) of tokens from x . It then estimates the token cohesiveness of x as:

$$u(x) = \sum_{i=1}^n \frac{\text{DIFF}(x, \tilde{x}_i)}{n}, \quad (1)$$

where $\text{DIFF}(\cdot, \cdot)$ measures the semantic difference between x and each of its copies $\tilde{x}_i$. We employ the established negative BART score (Yuan et al., 2021) as the metric, the details of which can be found in Appendix A. Meanwhile, we also feed the input x into another channel, which is equipped with an existing zero-shot detector, to produce a raw prediction $v(x)$. The output of the two channels are then combined into a new prediction:

$$w(x) = e^{u(x)} \times v(x), \quad (2)$$

on which we perform thresholding to make the final decision, i.e., x is categorized as LLM-generated if $w(x) > \epsilon$ or otherwise human-written. This detection procedure is summarized into Algorithm 1.

The proposed dual-channel detection paradigm is rather generic, allowing token cohesiveness to be applied as a plug-and-play module in a variety of detectors to further improve their performance. This paper chooses LRR (Su et al., 2023) and Fast-DetectGPT (Bao et al., 2024) as base detectors to verify the effectiveness and generality of TOCSIN. The details of the two base detectors are provided in Appendix B.

4 Experiments

This section evaluates the effectiveness of TOCSIN for zero-shot LLM-generated text detection compared with prior state-of-the-arts, and also provides extensive additional analyses to better understand multiple facets of the proposed method.

4.1 Experimental Setups

Datasets To ensure fair comparison, we follow prior work (Bao et al., 2024) to use four diversified datasets: *XSum* for news articles (Narayan et al., 2018), *SQuAD* for Wikipedia content (Rajpurkar et al., 2016), *WritingPrompts* for storytelling (Fan et al., 2018), and *PubMedQA* for biomedical question answering (Jin et al., 2019). Each dataset contains 150 to 500 randomly sampled human-written passages as negative samples, and the same number of LLM-generated passages as positive samples, created by prompting a source model with the first 30 tokens of each negative sample. Seven different source models of various size are considered, including the 1.5B *GPT-2* (Radford et al., 2019), 2.7B *OPT-2.7* (Zhang et al., 2022), 2.7B *GPT-Neo-2.7* (Black et al., 2021), 6B *GPT-J* (Wang and Komatsuzaki, 2021), 20B *GPT-NeoX* (Black et al., 2022), as well as the 175B *ChatGPT* (OpenAI, 2022) and the most powerful *GPT-4* (OpenAI, 2023) to simulate text generation in real-world scenarios. All the datasets are collected from the open-source project of Fast-DetectGPT (Bao et al., 2024).

Metric We also follow prior work to use the area under the receiver operating characteristic curve (AUROC) as the evaluation metric. This metric is based on dynamic positive-negative thresholds and does not require a fixed threshold to be specified.

Baselines As we have discussed in Section 3.3, TOCSIN is a generic paradigm that can be applied to various zero-shot detectors to further improve their performance. We choose two representative zero-shot detectors, *LRR* (log-probability log-rank ratio) (Su et al., 2023) and *Fast-DetectGPT* (conditional probability curvature) (Bao et al., 2024), and apply TOCSIN on the two detectors to validate the effectiveness and generality of our approach. We choose the two detectors as they are recently proposed, computationally efficient, and report current state-of-the-art performance. The derived methods are denoted as *LRR+TOCSIN* and *Fast-DetectGPT+TOCSIN*, respectively.

Besides the two direct baselines, we also compare with other established zero-shot detectors, including *Entropy* (entropy of predictive distribution), *Likelihood* (average log-probability), *LogRank* (average log-rank) (Mitchell et al., 2023), *NPR* (normalized log-rank of perturbations) (Su et al., 2023), *Detect-GPT* (probability curvature) (Mitchell et al., 2023), *DNA-GPT* (divergent n-grams in rewrites) (Yang et al., 2024). For completeness we also perform comparisons to existing supervised classifiers, including the GPT-2 detectors based on *RoBERTa-base/large* (Liu et al., 2019) crafted by OpenAI and *GPTZero* (Tian and Cui, 2023).

Implementation TOCSIN is conceptually simple and easy to implement. For the base detectors, we employ the code of *LRR*² and *Fast-DetectGPT*³, and keep their settings unchanged. The details of the two base detectors can be found in Appendix B. For token cohesiveness calculation, there are two hyperparameters: n (the number of copies created for each input) and ρ (the proportion of tokens to be deleted in each copy). We empirically set $n = 10$, $\rho = 1.5\%$ for **all experiments without re-tuning**, unless otherwise specified.

4.2 Experiments with Open-Source LLMs

We first evaluate TOCSIN in detecting text generated by the five open-source LLMs, from GPT-2 (1.5B) to GPT-NeoX (20B). We use 500 human samples along with 500 LLM samples generated by each of the five models across *XSum*, *SQuAD*, and *WritingPrompts*. We consider two evaluation settings: (1) a *white-box setting* where each source model is used to score passages, and (2) a *black-box setting* where the source models are not accessible and a surrogate model, i.e., GPT-Neo-2.7, is used to score passages. Note that this scoring is required by the base detectors, not by the token cohesiveness calculation module. Table 1 presents the AUROC scores averaged over the three datasets in the two settings, with detailed per dataset results reported in Table 4 in Appendix C.1.

Results in White-Box Setting As we can see, TOCSIN performs particularly well in this setting. It outperforms the direct baselines *LRR* and *Fast-DetectGPT* (as well as all the other baselines), irrespective of which dataset or source model is used. For *LRR*, the absolute improvement reaches 5.88%

²<https://github.com/mbzuai-nlp/DetectLLM>

³<https://github.com/baoguangsheng/fast-detect-gpt/>

Method	GPT-2	OPT-2.7	Neo-2.7	GPT-J	NeoX	Avg.
The White-Box Setting						
Entropy (Mitchell et al., 2023)	0.5174	0.4830	0.4898	0.5005	0.5333	0.5048
Likelihood (Mitchell et al., 2023)	0.9125	0.8963	0.8900	0.8480	0.7946	0.8683
LogRank (Mitchell et al., 2023)	0.9385	0.9223	0.9226	0.8818	0.8313	0.8993
DNA-GPT (Yang et al., 2024) [†]	0.9024	0.8797	0.8690	0.8227	0.7826	0.8513
DetectGPT (Mitchell et al., 2023) [†]	0.9917	0.9758	0.9797	0.9353	0.8943	0.9554
NPR (Su et al., 2023) [†]	0.9948	0.9832	0.9883	0.9500	0.9065	0.9645
LRR (Su et al., 2023)	0.9601	0.9401	0.9522	0.9179	0.8793	0.9299
LRR+TOCSIN (ours)	0.9919	0.9929	0.9873	0.9914	0.9799	0.9887
(Absolute $\uparrow$)	3.18%	5.28%	3.51%	7.35%	10.06%	5.88%
Fast-DetectGPT (Bao et al., 2024)	0.9967	0.9908	0.9940	0.9866	0.9754	0.9887
Fast-DetectGPT+TOCSIN (ours)	0.9986	0.9960	0.9978	0.9941	0.9863	0.9946
(Absolute $\uparrow$)	0.19%	0.52%	0.38%	0.75%	1.09%	0.59%
The Black-Box Setting						
DetectGPT (Mitchell et al., 2023) [†]	0.8517	0.8390	0.9797	0.8575	0.8400	0.8736
LRR (ours)	0.8505	0.8609	0.9518	0.8637	0.8187	0.8691
LRR+TOCSIN (ours)	0.9745	0.9849	0.9873	0.9832	0.9752	0.9810
(Absolute $\uparrow$)	12.40%	12.40%	3.55%	11.95%	15.65%	11.19%
Fast-DetectGPT (Bao et al., 2024)	0.9834	0.9572	0.9984	0.9592	0.9404	0.9677
Fast-DetectGPT+TOCSIN (ours)	0.9948	0.9815	0.9994	0.9822	0.9741	0.9864
(Absolute $\uparrow$)	1.14%	2.43%	0.10%	2.30%	3.37%	1.87%

Table 1: AUROC for zero-shot detection of passages generated from five source models, averaged across XSum, SQuAD, and WritingPrompts, with detailed results provided in Table 4 in Appendix C.1. Results marked by “(ours)” are produced by ourselves, and other results are taken directly from (Bao et al., 2024) to avoid implementation bias. In the white-box (resp. black-box) setting, the source model (resp. GPT-Neo-2.7) is used for scoring. [†] denotes methods that invoke scoring models multiple times, thereby significantly increasing computational demands. **Bold** indicates that a +TOCSIN variant outperforms its direct baseline, and “(Absolute $\uparrow$)” the absolute improvements.

in average AUROC. For Fast-DetectGPT, which already gets a super high average AUROC of 0.9887, TOCSIN still brings consistent and meaningful improvements, pushing the average AUROC further to 0.9946.

Results in Black-Box Setting In this setting we observe similar phenomena. TOCSIN, again, consistently outperforms all the baselines. Notably, the improvements are even more significant than those in the white-box setting (the absolute improvement in average AUROC reaches 11.19% over LRR and 1.87% over Fast-DetectGPT). We think this is because existing zero-shot detectors such as LRR and Fast-DetectGPT require the source model to make predictions. In the black-box setting where the source model is not available, they have to use a surrogate model for approximation, which inevitably results in performance degradation. TOCSIN, in contrast, requires no such approximation and therefore can resist the degradation.

4.3 Experiments with API-based LLMs

We further evaluate TOCSIN in detecting passages generated by ChatGPT and GPT-4 to simulate real-world scenarios. We use 150 human samples and 150 LLM samples generated by the two models

across XSum, WritingPrompts, and PubMedQA. As we are not able to access the source models, we consider the black-box setting, with GPT-Neo-2.7 used as the surrogate model. Table 2 reports averaged and per dataset AUROC, from which we can see that TOCSIN performs quite well in most cases. Compared to LRR, the absolute improvements in average AUROC reach 4.68% w.r.t. ChatGPT generated text and 9.42% w.r.t. GPT-4 generated text. Compared to Fast-DetectGPT which already performs well and even surpasses supervised classifiers, TOCSIN still brings meaningful overall improvements, i.e., 0.33% w.r.t. ChatGPT generated text and 1.57% w.r.t. GPT-4 generated text.

The only exception on which TOCSIN fails to achieve improvements is the PubMedQA dataset. We examine the dataset and find that PubMedQA passages, which consist of only 64 tokens on average, are substantially shorter than passages on the other datasets (e.g., 221 tokens on XSum and 218 tokens on WritingPrompts). For LLM-generated text, the generation of each token is an aggregation process, making the generated token more closely related to its preceding tokens. Passages of shorter length undergo fewer such aggregation processes, which may suppress the closeness of tokens therein and consequently reduce token cohesiveness, mak-

Method	ChatGPT				GPT-4			
	XSum	Writing	PubMed	Avg.	XSum	Writing	PubMed	Avg.
Supervised Classifiers								
RoBERTa-base (Bao et al., 2024)	0.9150	0.7084	0.6188	0.7474	0.6778	0.5068	0.5309	0.5718
RoBERTa-large (Bao et al., 2024)	0.8507	0.5480	0.6731	0.6906	0.6879	0.3821	0.6067	0.5589
GPTzero (Tian and Cui, 2023)	0.9952	0.9292	0.8799	0.9348	0.9815	0.8262	0.8482	0.8853
Zero-Shot Classifiers								
Entropy (Mitchell et al., 2023)	0.3305	0.1902	0.2767	0.2658	0.4360	0.3702	0.3295	0.3786
Likelihood (Mitchell et al., 2023)	0.9578	0.9740	0.8775	0.9364	0.7980	0.8553	0.8104	0.8212
LogRank (Mitchell et al., 2023)	0.9582	0.9656	0.8687	0.9308	0.7975	0.8286	0.8003	0.8088
DNA-GPT (Yang et al., 2024)†	0.9124	0.9425	0.7959	0.8836	0.7347	0.8032	0.7565	0.7648
DetectGPT (Mitchell et al., 2023)†	0.8416	0.8811	0.7444	0.8223	0.5660	0.6217	0.6805	0.6228
NPR (Su et al., 2023)†	0.7899	0.8924	0.6784	0.7869	0.5280	0.6122	0.6328	0.5910
LRR (Su et al., 2023)	0.9162	0.8958	0.7433	0.8518	0.7447	0.7028	0.6814	0.7096
LRR+TOCSIN (ours)	0.9939	0.9927	0.7092	0.8986	0.9614	0.8036	0.6465	0.8038
(Absolute ↑)	7.77%	9.69%	-3.41%	4.68%	21.67%	10.08%	-3.49%	9.42%
Fast-DetectGPT (Bao et al., 2024)	0.9907	0.9916	0.9021	0.9615	0.9067	0.9612	0.8503	0.9061
Fast-DetectGPT+TOCSIN (ours)	0.9969	0.9964	0.9011	0.9648	0.9455	0.9708	0.8490	0.9218
(Absolute ↑)	0.62%	0.48%	-0.10%	0.33%	3.88%	0.96%	-0.13%	1.57%

Table 2: AUROC for detecting passages generated by ChatGPT and GPT-4. Results marked by “(ours)” are produced by ourselves, and other results are taken directly from (Bao et al., 2024) to avoid implementation bias. The black-box setting is used for all zero-shot classifiers, with GPT-Neo-2.7 as the surrogate model. † denotes methods that invoke scoring models multiple times, thereby significantly increasing computational demands. **Bold** stands for better performance between a baseline and its +TOCSIN version, and “(Absolute ↑)” the absolute improvements.

ing it more difficult to distinguish these passages from human-written ones via token cohesiveness. We will show later in Figure 4 and Figure 5 the detailed impact of passage length.

4.4 Additional Analyses

We provide additional analyses to better understand multiple facets of TOCSIN. We take the best performing baseline Fast-DetectGPT as the base detector, and consider only the black-box setting where the source LLMs are not accessible.

Time & Space Efficiency As an improvement to existing zero-shot detectors, TOCSIN keeps the base detector unchanged, and the additional time and space costs mainly come from the computation of token cohesiveness, which involves 10 rounds of random token deletion and BART score computation. To rigorously assess the additional costs, we conduct time and space analysis on XSum, SQuAD, and WritingPrompts where the LLM-generated passages are produced by the five open-source models, and we make comparison between Fast-DetectGPT and Fast-DetectGPT+TOCSIN. All experiments here are conducted on a single NVIDIA A40 GPU with 48GB memory. Table 3 reports the runtime per instance and GPU memory usage of the two variants, averaged across the five source models and three datasets, with detailed per dataset and source model results further provided Appendix C.2. We

	Runtime (s)	GPU Memory (GB)
w/o TOCSIN	0.31	23.96
w/ TOCSIN	0.47	28.67
(Absolute ↑)	0.16	4.71

Table 3: Runtime per instance and GPU memory usage of w/ and w/o TOCSIN variants of Fast-DetectGPT in black-box setting, averaged across five source models on XSum, SQuAD, and WritingPrompts. “(Absolute ↑)” means additional time/space cost brought by TOCSIN.

can see that compared to the base detector, TOCSIN indeed brings a relatively low addition runtime of 0.16s per instance and a relatively low additional GPU memory usage of 4.71GB on average.

Impact of Hyperparameters TOCSIN gets two hyperparameters: the number of copies created for each input (n) and the proportion of tokens deleted in each copy (ρ). We examine the impact of the two hyperparameters, and show how the performance of Fast-DetectGPT+TOCSIN varies as $n \in \{10, 20, 50, 100\}$ and $\rho \in \{1.5\%, 5.0\%, 7.5\%, 10.0\%\}$. Figure 3 presents the results on XSum, SQuAD, and WritingPrompts with LLM-generated passages produced by the five open-source models, where $n = 0$ and $\rho = 0.0\%$ denote the performance of the base detector Fast-DetectGPT. We can see that the performance of TOCSIN is not sensitive to the hyperparameters. As for the number of copies n , TOCSIN performs rather stably with different n

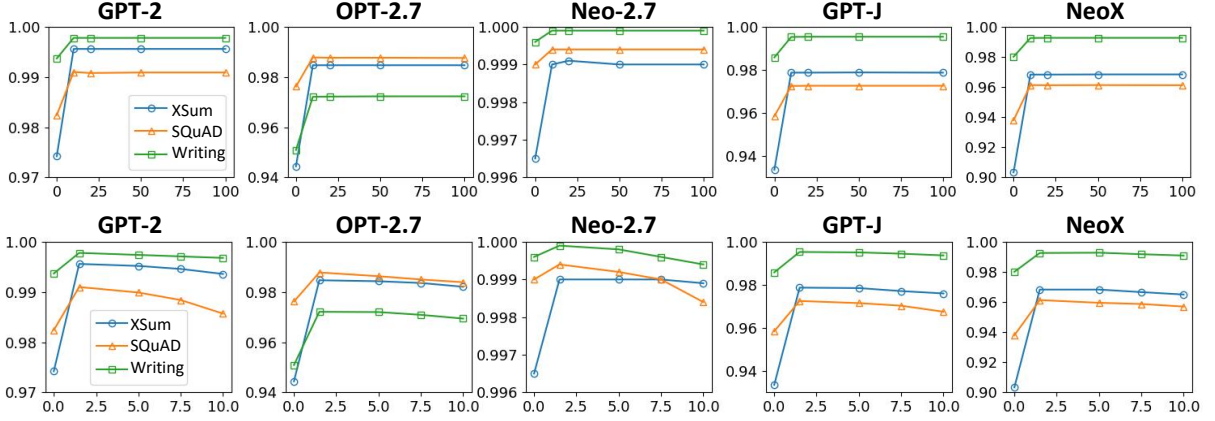


Figure 3: AUROC of FastDetectGPT+TOCSIN with varying hyperparameters on XSum, SQuAD, and WritingPrompts across five open-source LLMs in the black-box setting. **Top:** number of copies $n \in \{10, 20, 50, 100\}$ and token deletion proportion $\rho = 1.5\%$ where $n = 0$ denotes AUROC of Fast-DetectGPT. **Bottom:** $\rho \in \{1.5\%, 5.0\%, 7.5\%, 10.0\%\}$ and $n = 10$ where $\rho = 0.0\%$ denotes AUROC of Fast-DetectGPT.

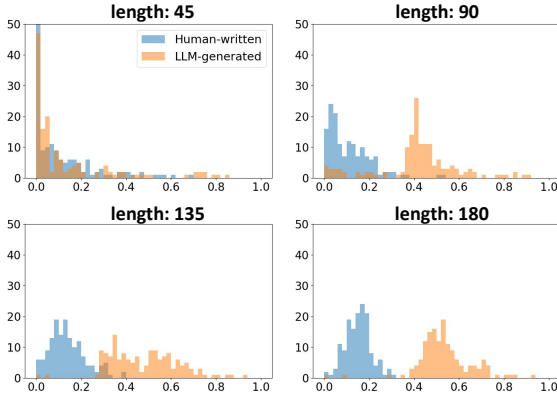


Figure 4: Distribution of token cohesiveness between 150 human-written and 150 LLM-generated passages from WritingPrompts truncated to target length. LLM-generated passages are produced by ChatGPT.

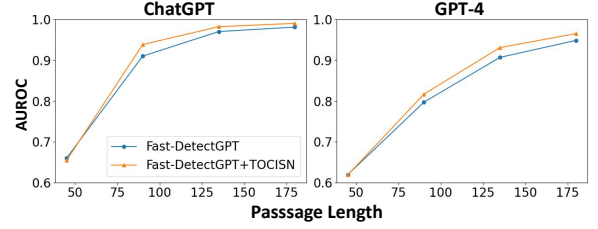


Figure 5: AUROC for black-box detection of passages generated by ChatGPT and GPT-4 on WritingPrompts truncated to target length.

values, so we just use $n = 10$ for simplicity and efficiency. As for the token deletion proportion ρ , a smaller ρ value of 1.5% generally performs better, and the performance drops slightly as ρ grows up.

Impact of Passage Length We have observed in Table 2 that TOCSIN may not perform that well on shorter passages. To rigorously evaluate the impact of passage length, we truncate the passages from WritingPrompts to various target lengths of 45, 90, 135, 180, and explore how the token cohesiveness and overall performance of TOCSIN varies at the four different lengths. The results are reported in Figure 4 and Figure 5. We can see that at a shorter length of 45, the distributions of token cohesiveness between human-written and LLM-generated text overlap significantly and cannot be discriminated. TOCSIN also fails to improve FastDetect-GPT at

this passage length. But as the length increases, the disparities in token cohesiveness between the two types of text become more obvious, and incorporating token cohesiveness into FastDetect-GPT starts to achieve consistent improvements. These results suggest that TOCSIN is more suitable for detecting long passages generated by LLMs.

5 Conclusion

This paper introduces the concept of token cohesiveness and unveils that it can serve as a new criterion to discriminate LLM-generated and human-written text. Based on this new finding, we devise TOCSIN for zero-shot detection of LLM-generated text. TOCSIN is a generic dual-channel detection paradigm that uses token cohesiveness as a plug-and-play module to improve existing zero-shot detectors. As empirical evaluation, we apply TOCSIN to two current state-of-the-art base detectors, and achieve meaningful improvements across four diversified datasets with passages generated from seven different source LLMs, demonstrating the effectiveness and generality of our approach.

553 Limitations

554 This work has two limitations. First, the proposed
 555 method TOCSIN, like most zero-shot detectors, is
 556 more suitable for long text and has limited effec-
 557 tiveness on short text. For example, it consistently
 558 performs well on passages consisting of 90 tokens
 559 or more, but fails on short passages with only 45 to-
 560 kens, as illustrated in Figure 5. Second, this work is
 561 restricted to the most basic form of LLM-generated
 562 text detection, i.e., binary classification of LLM-
 563 generated and human-written text. Whether TOC-
 564 SIN still works in more challenging tasks, such as
 565 detecting mixtures of LLM-generated and human-
 566 written text and tracing the origin of generation,
 567 remains an open question for future research.

568 References

569 Anton Bakhtin, Sam Gross, Myle Ott, Yuntian Deng,
 570 Marc’Aurelio Ranzato, and Arthur Szlam. 2019.
 571 [Real or fake? Learning to discriminate ma-
 572 chine from human generated text.](#) *arXiv preprint
 573 arXiv:1906.03351*.

574 Guangsheng Bao, Yanbin Zhao, Zhiyang Teng, Linyi
 575 Yang, and Yue Zhang. 2024. [Fast-detectGPT: Effi-
 576 cient zero-shot detection of machine-generated text
 577 via conditional probability curvature.](#) In *The Twelfth
 578 International Conference on Learning Representa-
 579 tions*.

580 Amrita Bhattacharjee, Tharindu Kumarage, Raha
 581 Moraffah, and Huan Liu. 2023. [ConDA: Contrastive
 582 domain adaptation for AI-generated text detection.](#)
 583 In *Proceedings of the 13th International Joint Con-
 584 ference on Natural Language Processing and the 3rd
 585 Conference of the Asia-Pacific Chapter of the Associ-
 586 ation for Computational Linguistics*, pages 598–610.

587 Sid Black, Leo Gao, Phil Wang, Connor Leahy, and
 588 Stella Biderman. 2021. [GPT-Neo: Large scale autore-
 589 gressive language modeling with Mesh-Tensorflow.](#)
 590 <https://github.com/EleutherAI/gpt-neo>.

591 Sidney Black, Stella Biderman, Eric Hallahan, Quentin
 592 Anthony, Leo Gao, Laurence Golding, Horace He,
 593 Connor Leahy, Kyle McDonell, Jason Phang, et al.
 594 2022. [GPT-NeoX-20B: An open-source autoregres-
 595 sive language model.](#) In *Proceedings of BigScience
 596 Episode #5 – Workshop on Challenges & Perspec-
 597 tives in Creating Large Language Models*, pages 95–
 598 136.

599 Yutian Chen, Hao Kang, Vivian Zhai, Liangze Li, Rita
 600 Singh, and Bhiksha Raj. 2023. [GPT-sentinel: Dis-
 601 tinguishing human and ChatGPT generated content.](#)
 602 *arXiv preprint arXiv:2305.07969*.

603 Aakanksha Chowdhery, Sharan Narang, Jacob Devlin,
 604 Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul

605 Barham, Hyung Won Chung, Charles Sutton, Sebas-
 606 tian Gehrmann, et al. 2023. [PaLM: Scaling language
 607 modeling with pathways.](#) *Journal of Machine Learn-
 608 ing Research*, 24(240):1–113.

609 Jacob Devlin, Ming-Wei Chang, Kenton Lee, and
 610 Kristina Toutanova. 2019. [BERT: Pre-training of
 611 deep bidirectional transformers for language under-
 612 standing.](#) In *Proceedings of the 2019 Conference
 613 of the North American Chapter of the Association
 614 for Computational Linguistics: Human Language
 615 Technologies*, pages 4171–4186.

616 Tiziano Fagni, Fabrizio Falchi, Margherita Gambini, An-
 617 tonio Martella, and Maurizio Tesconi. 2021. [Tweep-
 618 fake: About detecting deepfake tweets.](#) *PLOS ONE*,
 619 16(5):e0251415.

620 Angela Fan, Mike Lewis, and Yann Dauphin. 2018.
 621 [Hierarchical neural story generation.](#) In *Proceedings
 622 of the 56th Annual Meeting of the Association for
 623 Computational Linguistics*, pages 889–898.

624 Sebastian Gehrmann, Hendrik Strobelt, and Alexander
 625 Rush. 2019. [GLTR: Statistical detection and visual-
 626 ization of generated text.](#) In *Proceedings of the 57th
 627 Annual Meeting of the Association for Computational
 628 Linguistics: System Demonstrations*, pages 111–116.

629 Biyang Guo, Xin Zhang, Ziyuan Wang, Minqi Jiang,
 630 Jinran Nie, Yuxuan Ding, Jianwei Yue, and Yupeng
 631 Wu. 2023. [How close is ChatGPT to human experts?
 632 Comparison corpus, evaluation, and detection.](#) *arXiv
 633 preprint arXiv:2301.07597*.

634 Tatsunori B. Hashimoto, Hugh Zhang, and Percy Liang.
 635 2019. [Unifying human and statistical evaluation for
 636 natural language generation.](#) In *Proceedings of the
 637 2019 Conference of the North American Chapter of
 638 the Association for Computational Linguistics: Hu-
 639 man Language Technologies*, pages 1689–1701.

640 Daphne Ippolito, Daniel Duckworth, Chris Callison-
 641 Burch, and Douglas Eck. 2020. [Automatic detection
 642 of generated text is easiest when humans are fooled.](#)
 643 In *Proceedings of the 58th Annual Meeting of the As-
 644 sociation for Computational Linguistics*, pages 1808–
 645 1822.

646 Bohan Jiang, Zhen Tan, Ayushi Nirmal, and Huan Liu.
 647 2024. [Disinformation detection: An evolving chal-
 648 lenge in the age of LLMs.](#) In *Proceedings of the
 649 2024 SIAM International Conference on Data Min-
 650 ing*, pages 427–435.

651 Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Co-
 652 hen, and Xinghua Lu. 2019. [PubMedQA: A dataset
 653 for biomedical research question answering.](#) In *Pro-
 654 ceedings of the 2019 Conference on Empirical Meth-
 655 ods in Natural Language Processing and the 9th In-
 656 ternational Joint Conference on Natural Language
 657 Processing*, pages 2567–2577.

658 Thomas Lavergne, Tanguy Urvoy, and François Yvon.
 659 2008. [Detecting fake content with relative entropy
 660 scoring.](#) In *Proceedings of the 2008 International*

661	<i>Conference on Uncovering Plagiarism, Authorship</i>	714
662	<i>and Social Software Misuse</i> , volume 377, pages 27–	715
663	31.	716
664	Mike Lewis, Yinhan Liu, Naman Goyal, Marjan	717
665	Ghazvininejad, Abdelrahman Mohamed, Omer Levy,	718
666	Veselin Stoyanov, and Luke Zettlemoyer. 2020.	719
667	BART: Denoising sequence-to-sequence pre-training	
668	for natural language generation, translation, and com-	
669	prehension . In <i>Proceedings of the 58th Annual Meet-</i>	
670	<i>ing of the Association for Computational Linguistics</i> ,	
671	pages 7871–7880.	
672	Linyang Li, Pengyu Wang, Ke Ren, Tianxiang Sun, and	
673	Xipeng Qiu. 2023a. Origin tracing and detecting of	
674	LLMs . <i>arXiv preprint arXiv:2304.14072</i> .	
675	Yafu Li, Qintong Li, Leyang Cui, Wei Bi, Longyue	
676	Wang, Linyi Yang, Shuming Shi, and Yue Zhang.	
677	2023b. Deepfake text detection in the wild . <i>arXiv</i>	
678	<i>preprint arXiv:2305.13242</i> .	
679	Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Man-	
680	dar Joshi, Danqi Chen, Omer Levy, Mike Lewis,	
681	Luke Zettlemoyer, and Veselin Stoyanov. 2019.	
682	RoBERTa: A robustly optimized BERT pretraining	
683	approach . <i>arXiv preprint arXiv:1907.11692</i> .	
684	Eric Mitchell, Yoonho Lee, Alexander Khazatsky,	
685	Christopher D Manning, and Chelsea Finn. 2023. De-	
686	tectGPT: Zero-shot machine-generated text detection	
687	using probability curvature . In <i>Proceedings of the</i>	
688	<i>40th International Conference on Machine Learning</i> ,	
689	pages 24950–24962.	
690	Sandra Mitrović, Davide Andreoletti, and Omran Ayoub.	
691	2023. Chatgpt or human? Detect and explain. Ex-	
692	plaining decisions of machine learning model for de-	
693	tecting short ChatGPT-generated text . <i>arXiv preprint</i>	
694	<i>arXiv:2301.13852</i> .	
695	Shashi Narayan, Shay B. Cohen, and Mirella Lapata.	
696	2018. Don't give me the details, just the summary!	
697	Topic-aware convolutional neural networks for ex-	
698	treme summarization . In <i>Proceedings of the 2018</i>	
699	<i>Conference on Empirical Methods in Natural Lan-</i>	
700	<i>guage Processing</i> , pages 1797–1807.	
701	OpenAI. 2022. ChatGPT: Optimizing language models	
702	for dialogue . https://openai.com/blog/chatgpt .	
703	OpenAI. 2023. GPT-4 technical report . <i>arXiv preprint</i>	
704	<i>arXiv:2303.08774</i> .	
705	Ashwinee Panda, Christopher A. Choquette-Choo,	
706	Zhengming Zhang, Yaoqing Yang, and Prateek Mit-	
707	tal. 2024. Teach LLMs to phish: Stealing private	
708	information from language models . <i>arXiv preprint</i>	
709	<i>arXiv:2403.00871</i> .	
710	Mike Perkins. 2023. Academic integrity considerations	
711	of AI large language models in the post-pandemic	
712	era: ChatGPT and beyond . <i>Journal of University</i>	
713	<i>Teaching & Learning Practice</i> , 20(2):07.	
Jiameng Pu, Zain Sarwar, Sifat Muhammad Abdullah,		
Abdullah Rehman, Yoonjin Kim, Parantapa Bhat-		
tacharya, Mobin Javed, and Bimal Viswanath. 2023.		
Deepfake text detection: Limitations and opportu-		
nities . In <i>2023 IEEE Symposium on Security and</i>		
<i>Privacy</i> , pages 1613–1630.		
Alec Radford, Jeffrey Wu, Rewon Child, David Luan,		
Dario Amodei, and Ilya Sutskever. 2019. Language		
models are unsupervised multitask learners . <i>OpenAI</i>		
<i>blog</i> , 1(8):9.		
Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and		
Percy Liang. 2016. SQuAD: 100,000+ questions		
for machine comprehension of text . In <i>Proceedings</i>		
<i>of the 2016 Conference on Empirical Methods in</i>		
<i>Natural Language Processing</i> , pages 2383–2392.		
Juan Diego Rodriguez, Todd Hay, David Gros, Zain		
Shamsi, and Ravi Srinivasan. 2022. Cross-domain		
detection of GPT-2-generated technical text . In <i>Pro-</i>		
<i>ceedings of the 2022 Conference of the North Amer-</i>		
<i>ican Chapter of the Association for Computational</i>		
<i>Linguistics: Human Language Technologies</i> , pages		
1213–1233.		
Irene Solaiman, Miles Brundage, Jack Clark, Amanda		
Askell, Ariel Herbert-Voss, Jeff Wu, Alec Rad-		
ford, Gretchen Krueger, Jong Wook Kim, Sarah		
Kreps, et al. 2019. Release strategies and the so-		
cial impacts of language models . <i>arXiv preprint</i>		
<i>arXiv:1908.09203</i> .		
Jinyan Su, Terry Zhuo, Di Wang, and Preslav Nakov.		
2023. DetectLLM: Leveraging log rank information		
for zero-shot detection of machine-generated text .		
In <i>Findings of the Association for Computational</i>		
<i>Linguistics: EMNLP 2023</i> , pages 12395–12412.		
Ruixiang Tang, Yu-Neng Chuang, and Xia Hu. 2024.		
The science of detecting LLM-generated text . <i>Com-</i>		
<i>munications of the ACM</i> , 67(4):50–59.		
Edward Tian and Alexander Cui. 2023. GPTZero: To-		
wards detection of AI-generated text using zero-shot		
and supervised methods . https://gptzero.me .		
Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier		
Martinet, Marie-Anne Lachaux, Timothée Lacroix,		
Baptiste Rozière, Naman Goyal, Eric Hambro,		
Faisal Azhar, et al. 2023. LLaMA: Open and ef-		
ficient foundation language models . <i>arXiv preprint</i>		
<i>arXiv:2302.13971</i> .		
Adaku Uchendu, Thai Le, Kai Shu, and Dongwon Lee.		
2020. Authorship attribution for neural text gener-		
ation . In <i>Proceedings of the 2020 Conference on</i>		
<i>Empirical Methods in Natural Language Processing</i> ,		
pages 8384–8395.		
Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob		
Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz		
Kaiser, and Illia Polosukhin. 2017. Attention is all		
you need . In <i>Advances in Neural Information Pro-</i>		
<i>cessing Systems</i> , volume 30.		

Vivek Verma, Eve Fleisig, Nicholas Tomlin, and Dan Klein. 2023. [Ghostbuster: Detecting text ghost-written by large language models](#). *arXiv preprint arXiv:2305.15047*.

Ben Wang and Aran Komatsuzaki. 2021. [GPT-J-6B: A 6 billion parameter autoregressive language model](#). <https://github.com/kingoflolz/mesh-transformer-jax>.

Pengyu Wang, Linyang Li, Ke Ren, Botian Jiang, Dong Zhang, and Xipeng Qiu. 2023a. [SeqXGPT: Sentence-level AI-generated text detection](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1144–1156.

Quan Wang, Licheng Zhang, Zikang Guo, and Zhen-dong Mao. 2024. [IDEATE: Detecting AI-generated text using internal and external factual structures](#). In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 8556–8568.

Yuxia Wang, Jonibek Mansurov, Petar Ivanov, Jinyan Su, Artem Shelmanov, Akim Tsvigun, Chenxi Whitehouse, Osama Mohammed Afzal, Tarek Mahmoud, Alham Fikri Aji, et al. 2023b. [M4: Multi-generator, multi-domain, and multi-lingual black-box machine-generated text detection](#). *arXiv preprint arXiv:2305.14902*.

Kangxi Wu, Liang Pang, Huawei Shen, Xueqi Cheng, and Tat-Seng Chua. 2023. [LLMDet: A large language models detection tool](#). *arXiv preprint arXiv:2305.15004*.

Xianjun Yang, Wei Cheng, Yue Wu, Linda Ruth Petzold, William Yang Wang, and Haifeng Chen. 2024. [DNA-GPT: Divergent n-gram analysis for training-free detection of GPT-generated text](#). In *The Twelfth International Conference on Learning Representations*.

Weizhe Yuan, Graham Neubig, and Pengfei Liu. 2021. [BARTScore: Evaluating generated text as text generation](#). In *Advances in Neural Information Processing Systems*, volume 34, pages 27263–27277.

Rowan Zellers, Ari Holtzman, Hannah Rashkin, Yonatan Bisk, Ali Farhadi, Franziska Roesner, and Yejin Choi. 2019. [Defending against neural fake news](#). In *Advances in Neural Information Processing Systems*, volume 32.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. [OPT: Open pre-trained transformer language models](#). *arXiv preprint arXiv:2205.01068*.

Wanjun Zhong, Duyu Tang, Zenan Xu, Ruize Wang, Nan Duan, Ming Zhou, Jiahai Wang, and Jian Yin. 2020. [Neural deepfake detection with factual structure of text](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 2461–2470.

Biru Zhu, Lifan Yuan, Ganqu Cui, Yangyi Chen, Chong Fu, Bingxiang He, Yangdong Deng, Zhiyuan Liu, Maosong Sun, and Ming Gu. 2023. [Beat LLMs at their own game: Zero-shot LLM-generated text detection via querying ChatGPT](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7470–7483.

A BART Score Computation

We use the negative BART score (Yuan et al., 2021) to measure the semantic difference between input x and its copy $\tilde{x}$ with some tokens randomly deleted:

$$\text{DIFF}(x, \tilde{x}) = -\text{BARTScore}(x, \tilde{x}).$$

BART score, built on a BART model (Lewis et al., 2020) parameterized by θ , is calculated as the log probability of generating the input x (target text) conditioned on its copy $\tilde{x}$ (source text), which can be factorized as:

$$\text{BARTScore}(x, \tilde{x}) = \sum_{j=1}^k \log p_{\theta}(x_j | x_{<j}, \tilde{x}).$$

Here, k is the total number of tokens in x , x_j the j -th token therein, and $x_{<j}$ the sequence preceding x_j . This score essentially measures the semantic coverage between the source and target text, and its negative value therefore measures their semantic difference. In this paper, we use BART-base which has 139M parameters to compute the BART score, so as to ensure the efficiency of token cohesiveness calculation. Note that this model is much smaller than the surrogate model required by the base detector, e.g., the 2.7B GPT-Neo-2.7 used by LRR and Fast-DetectGPT.

B Base Detectors

In this paper we choose LRR (Su et al., 2023) and Fast-DetectGPT (Bao et al., 2024), which are computationally efficient and report current state-of-the-art performance as the base detectors.

LRR LRR makes use of the Log-Likelihood Log-Rank Ratio to discern LLM-generated and human-written text. Given a candidate passage x , LRR is formally defined as:

$$\text{LRR}(x) = -\frac{\sum_{j=1}^k \log p_{\theta}(x_j | x_{<j})}{\sum_{j=1}^k \log r_{\theta}(x_j | x_{<j})},$$

where $p_{\theta}(x_j | x_{<j})$ is the probability of token x_j conditioned on its preceding tokens predicted by

Dataset	Method	GPT-2	OPT-2.7	Neo-2.7	GPT-J	NeoX	Avg.
XSum	The White-Box Setting						
	Entropy (Mitchell et al., 2023)	0.5835	0.5071	0.5712	0.5705	0.6035	0.5671
	Likelihood (Mitchell et al., 2023)	0.8638	0.8600	0.8609	0.8101	0.7604	0.8310
	LogRank (Mitchell et al., 2023)	0.8918	0.8839	0.8949	0.8407	0.7939	0.8610
	DNA-GPT (Yang et al., 2024)†	0.8548	0.8168	0.8197	0.7586	0.7167	0.7933
	DetectGPT (Mitchell et al., 2023)†	0.9875	0.9621	0.9914	0.9632	0.9398	0.9688
	NPR (Su et al., 2023)†	0.9891	0.9681	0.9929	0.9566	0.9311	0.9676
	LRR (Su et al., 2023)	0.9179	0.8867	0.9190	0.8592	0.8205	0.8807
	LRR+TOCSIN (ours)	0.9957	0.9968	0.9935	0.9950	0.9926	0.9947
	(Absolute ↑)	7.78%	11.01%	7.45%	13.58%	17.21%	11.40%
	Fast-DetectGPT (Bao et al., 2024)	0.9930	0.9803	0.9885	0.9771	0.9703	0.9818
	Fast-DetectGPT+TOCSIN (ours)	0.9974	0.9928	0.9966	0.9927	0.9850	0.9929
	(Absolute ↑)	0.44%	1.25%	0.81%	1.56%	1.47%	1.11%
	The Black-Box Setting						
	DetectGPT (Mitchell et al., 2023)†	0.9180	0.8868	0.9914	0.8830	0.8682	0.9095
	LRR (ours)	0.7824	0.8069	0.9186	0.7767	0.7357	0.8041
	LRR+TOCSIN (ours)	0.9904	0.9953	0.9935	0.9914	0.9901	0.9921
	(Absolute ↑)	20.80%	18.84%	7.49%	21.47%	25.44%	18.80%
	Fast-DetectGPT (Bao et al., 2024)	0.9742	0.9444	0.9965	0.9335	0.9033	0.9504
	Fast-DetectGPT+TOCSIN (ours)	0.9956	0.9847	0.9990	0.9787	0.9683	0.9853
	(Absolute ↑)	2.14%	4.03%	0.25%	4.52%	6.50%	3.49%
SQuAD	The White-Box Setting						
	Entropy (Mitchell et al., 2023)	0.5791	0.5119	0.5581	0.5643	0.6056	0.5638
	Likelihood (Mitchell et al., 2023)	0.9077	0.8839	0.8585	0.7943	0.6977	0.8284
	LogRank (Mitchell et al., 2023)	0.9454	0.9203	0.9054	0.8471	0.7545	0.8745
	DNA-GPT (Yang et al., 2024)†	0.9094	0.8934	0.8589	0.8069	0.7525	0.8442
	DetectGPT (Mitchell et al., 2023)†	0.9914	0.9763	0.9625	0.8738	0.7916	0.9191
	NPR (Su et al., 2023)†	0.9965	0.9853	0.9789	0.9108	0.8175	0.9378
	LRR (Su et al., 2023)	0.9773	0.9597	0.9610	0.9244	0.8600	0.9365
	LRR+TOCSIN (ours)	0.9928	0.9836	0.9777	0.9862	0.9621	0.9805
	(Absolute ↑)	1.55%	2.39%	1.67%	6.18%	10.21%	4.40%
	Fast-DetectGPT (Bao et al., 2024)	0.9990	0.9949	0.9956	0.9853	0.9617	0.9873
	Fast-DetectGPT+TOCSIN (ours)	0.9996	0.9972	0.9975	0.9904	0.9764	0.9922
	(Absolute ↑)	0.06%	0.23%	0.19%	0.51%	1.47%	0.49%
	The Black-Box Setting						
	DetectGPT (Mitchell et al., 2023)†	0.7382	0.7530	0.9625	0.7882	0.7709	0.8026
	LRR (ours)	0.8447	0.8615	0.9603	0.8709	0.8109	0.8697
	LRR+TOCSIN (ours)	0.9713	0.9620	0.9777	0.9701	0.9552	0.9673
	(Absolute ↑)	12.66%	10.05%	1.74%	9.92%	14.43%	9.76%
	Fast-DetectGPT (Bao et al., 2024)	0.9824	0.9762	0.9990	0.9584	0.9379	0.9708
	Fast-DetectGPT+TOCSIN (ours)	0.9910	0.9878	0.9994	0.9725	0.9613	0.9824
	(Absolute ↑)	0.86%	1.16%	0.04%	1.41%	2.34%	1.16%
WritingPrompts	The White-Box Setting						
	Entropy (Mitchell et al., 2023)	0.3895	0.4299	0.3400	0.3668	0.3908	0.3834
	Likelihood (Mitchell et al., 2023)	0.9661	0.9451	0.9505	0.9396	0.9256	0.9454
	LogRank (Mitchell et al., 2023)	0.9782	0.9628	0.9675	0.9577	0.9454	0.9623
	DNA-GPT (Yang et al., 2024)†	0.9431	0.9288	0.9283	0.9026	0.8786	0.9163
	DetectGPT (Mitchell et al., 2023)†	0.9962	0.9891	0.9852	0.9688	0.9516	0.9782
	NPR (Su et al., 2023)†	0.9987	0.9962	0.9930	0.9825	0.9708	0.9882
	LRR (Su et al., 2023)	0.9850	0.9740	0.9766	0.9702	0.9573	0.9726
	LRR+TOCSIN (ours)	0.9871	0.9983	0.9907	0.9929	0.9850	0.9908
	(Absolute ↑)	0.21%	2.43%	1.41%	2.27%	2.77%	1.82%
	Fast-DetectGPT (Bao et al., 2024)	0.9982	0.9972	0.9980	0.9974	0.9941	0.9970
	Fast-DetectGPT+TOCSIN (ours)	0.9988	0.9979	0.9993	0.9992	0.9974	0.9985
	(Absolute ↑)	0.06%	0.07%	0.13%	0.18%	0.33%	0.15%
	The Black-Box Setting						
	DetectGPT (Mitchell et al., 2023)†	0.8989	0.8772	0.9852	0.9014	0.8809	0.9087
	LRR (ours)	0.9244	0.9142	0.9766	0.9436	0.9095	0.9337
	LRR+TOCSIN (ours)	0.9617	0.9975	0.9907	0.9880	0.9803	0.9836
	(Absolute ↑)	3.73%	8.33%	1.41%	4.44%	7.08%	4.99%
	Fast-DetectGPT (Bao et al., 2024)	0.9937	0.9509	0.9996	0.9858	0.9801	0.9820
	Fast-DetectGPT+TOCSIN (ours)	0.9978	0.9721	0.9999	0.9953	0.9926	0.9915
	(Absolute ↑)	0.41%	2.12%	0.03%	0.95%	1.25%	0.95%

Table 4: Details of the main results in Table 1 on three datasets, with all settings identical to those in Table 1.

Dataset	Method	Runtime (s)					GPU Memory (GB)				
		GPT-2	OPT-2.7	Neo-2.7	GPT-J	NeoX	GPT-2	OPT-2.7	Neo-2.7	GPT-J	NeoX
XSum	w/o TOCSIN	0.31	0.31	0.31	0.31	0.31	23.57	23.30	23.22	24.21	23.53
	w/ TOCSIN	0.48	0.46	0.46	0.47	0.46	28.35	27.80	28.37	28.79	27.35
	(Absolute ↑)	0.17	0.15	0.15	0.16	0.15	4.78	4.50	5.15	4.58	3.82
SQuAD	w/o TOCSIN	0.32	0.32	0.32	0.32	0.31	23.35	23.98	23.90	24.05	24.00
	w/ TOCSIN	0.49	0.46	0.49	0.48	0.47	28.14	28.53	28.54	28.07	29.19
	(Absolute ↑)	0.17	0.14	0.17	0.16	0.16	4.79	4.55	4.64	4.02	5.19
Writing	w/o TOCSIN	0.32	0.30	0.31	0.31	0.31	24.98	24.30	23.89	24.75	24.35
	w/ TOCSIN	0.48	0.45	0.48	0.47	0.47	29.31	29.24	29.51	29.56	29.32
	(Absolute ↑)	0.16	0.15	0.17	0.16	0.16	4.33	4.94	5.62	4.81	4.97

Table 5: Runtime per instance and GPU memory usage of w/ and w/o TOCSIN variants of Fast-DetectGPT in black-box setting. Results are across five source models on three datasets. “(Absolute ↑)” means additional time/space cost brought by TOCSIN.

model θ , and $r_\theta(x_j|x_{<j})$ the rank of that probability. Model θ is the source LLM in the white-box setting and the surrogate model in the black-box setting (GPT-Neo-2.7 in this paper). LLM-generated text is supposed to have larger LRR compared to human-written text.

Fast-DetectGPT Fast-DetectGPT makes uses of a conditional probability function to detect LLM-generated text. Given a passage x , it first performs conditional independent sampling from $q_\psi(\cdot|x)$ to create a group of samples $\{\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_n\}$. This sampling samples alternative word choices at each token conditioned on the fixed passage x without depending on other sampled tokens. Then, it evaluates the conditional probabilities $p_\theta(\tilde{x}|x)$ of these samples and combines them to arrive at a decision:

$$\text{Fast-DetectGPT}(x) = \frac{1}{n} \sum_{i=1}^n \log \frac{p_\theta(x|x)}{p_\theta(\tilde{x}|x)}.$$

If the score exceeds a specific threshold, the passage is probably LLM-generated. Fast-DetectGPT invokes the sampling model $q_\psi(\cdot|x)$ once to generate all samples and similarly the scoring model $p_\theta(\cdot|x)$ once to evaluate all samples, and therefore is rather efficient. In the white-box setting, $q_\psi(\cdot|x)$ and $p_\theta(\cdot|x)$ are both set to the source LLM. In the black-box setting, $q_\psi(\cdot|x)$ is set to the 6B GPT-J and $p_\theta(\cdot|x)$ the 2.7B GPT-Neo-2.7. All configurations are identical to those in (Bao et al., 2024).

C Additional Experimental Results

C.1 Detailed Results of Open-Source LLMs

Table 4 presents AUROC for zero-shot detection of passages generated by the five open-source LLMs on XSum, SQuAD, and WritingPrompts. TOCSIN

brings consistent improvements in both white-box and black-box settings, regardless of the datasets, source models, or base detectors.

C.2 Details of Time & Space Efficiency

Table 5 reports detailed time/space analysis results of Fast-DetectGPT and Fast-DetectGPT+TOCSIN in black-box setting on XSum, SQuAD, and WritingPrompts, with passages generated by the five open-source LLMs. As TOCSIN always performs 10 rounds of random token deletion and uses BART-base to calculate token cohesiveness without any other requirements, the additional time/space costs are rather stable across datasets and source models (as long as the passages are roughly of equal size). On average, TOCSIN bring an additional runtime of 0.16s per instance and additional GPU memory usage of 4.71GB.